Manifesto:
Gate-Driven Development

Marcela Gotta, November 12, 2026. v0.10, draft for review.

Add gates to your project and automate with AI agents without losing quality. Your agent says "done." The gate checks if it's true.

Gate-Driven Development (GDD) is a way to build quality products with AI agents without supervising them step by step. You trust what they prove, not what they say: before the agent starts, you decide what it has to prove, and those are the gates. The more that can go wrong, the stricter they are. When something slips through, you harden the gate so it doesn't happen again, and when one no longer adds anything, you relax it or retire it.

All of GDD fits in one rule (your agent says "done," the gate checks if it's true), seven principles and four moments.

The problem

Agents write fast and almost right. If you review everything, you become the slowest step. If you review nothing, the errors reach production. And if the agent can touch what judges it, it learns to approve itself.

In the end it is an untamable beast, and you don't persuade a beast like that with words: you put gates on it.

That is the basis of GDD: **we don't develop the specifications, or the code, or even the tests. We design the gates our agents will go through.**

I explain how I got here in the Article.

The pieces

- **Gate:** what has to be proven before moving on.
- **Checks:** what proves it, one or several, of three types: machine (tests, linter, Sonar), reviewer (another agent that runs the change and only passes the change if it checks that it works) and person (your approval). You get through the gate only if all of them pass.
- **Context:** what you give the agent so it can get through the gate, whether it's in the repository or comes from outside. The agent always starts with a clean context, with only what it needs: the cross-cutting context, which applies to the whole flow (the project's constitution, meaning its rules, and the usual tools and skills), and the context of its gate, which can include what the previous gate generated. It helps it get through, but it proves nothing.
- **Risk:** what can go wrong.
- **Blueprint:** where it's written down what is delicate, which gates are needed, with their checks, and what each level of risk asks for. When a check can be relaxed, and what would make you retire a gate, is decided here too, not in the change. You approve it, or your team if you work with one, and the agent doesn't touch it.

An agent never takes the place of a person. You always approve the blueprint, or your team does, and the blueprint says where else an approval is needed, including the approval of what has to pass. And your approval covers what you saw: if it changes afterwards, you are asked again.

Risk is how much damage a change can do to your product. I read it as a traffic light, and each color is a level of strictness:

Risk	For example
🟢 **Green** (low)	Changing some text

Risk	For example
🟡 **Yellow** (medium)	A new filter on a list
🔴 **Red** (high)	Something that touches payment

What each level asks for is your decision, or your team's, and it's written in the blueprint: how many checks, of which type, and at which moments it waits for you. In some projects a green change ships without anyone looking, and in others even green waits for an approval. What holds in all of them is that more risk asks for more. So you don't start from scratch, Good practices and the ready-made rules suggest what to ask for at each level, by area.

The blueprint brings the base of what is delicate, and the risk of each change is read against it. Green is only what you have marked as green. The rest isn't red just like that: an agent that won't do the work analyzes it and writes down why, and if it isn't sure, it asks you. What it notes down isn't the blueprint: it proposes it to you, and the blueprint only changes with your yes.

What can't be undone is never green. And what each level asks for has a single exception, which doesn't depend on your blueprint: on red, nothing ships without a verified way to undo it, and if there isn't one, a person decides knowing that.

The seven principles

The pieces say what GDD is made of. The principles say what makes it work.

1. **Gates come first, and nobody skips them, not even you.** They are defined before the work. If you decide what to check once you can see the result, you end up checking what the agent did and not

what you needed. Emergencies have a gate of their own, decided beforehand: a person opens it, it's written down, and the change goes through the others afterwards.

2 **A gate that never closes is not a gate.** It's a drawing on the wall. Before you trust a gate, you check that it closes, and that it closes for the right reason.

3 **To each gate, its context.** A gate without context is a wall: the agent guesses, fails and guesses again. With too much it also gets lost and costs more, so on top of the cross-cutting context, which always goes along, you give it just what it needs for that gate. It helps it get through, but it proves nothing.

4 **You get through a gate with facts, not words.** Saying "done" opens nothing. Without facts it doesn't get through, and if it gets stuck, it stops and asks.

5 **Whoever goes through the gate doesn't decide how it's validated.** The agent never touches the tests that judge it, just as a student doesn't grade their own exam. If a check is wrong, whoever wrote it or a person corrects it, never whoever has to pass it.

6 **More risk, more checks, and your approval only where it matters.** The size of the change doesn't matter: what can go wrong does. More risk isn't always more tests: often it is another type of check, like a reviewer or a person. You approve where your blueprint says so. The gates handle the rest, because someone who has to approve everything ends up approving without reading.

7 **Gates with cracks get hardened.** What slips through, or keeps repeating, becomes a check and a question on the reviewer's exam. And hardening is also knowing when to reduce: a gate that no

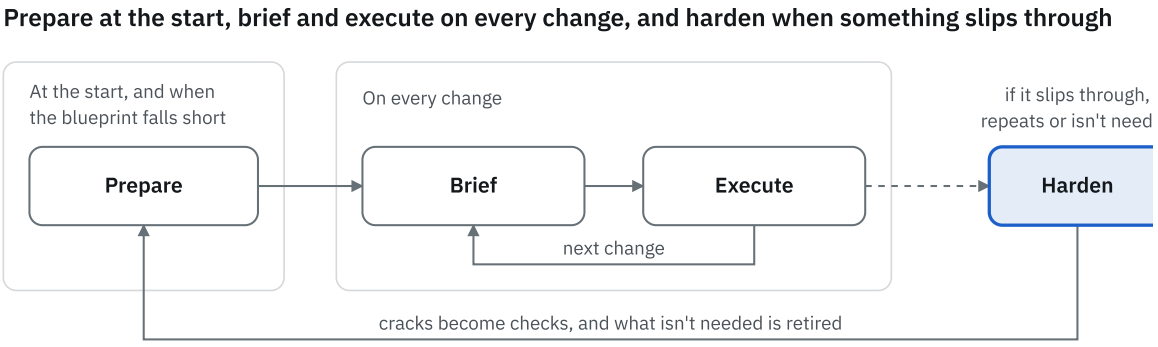
> longer catches anything, or that costs more than it protects, is relaxed or retired, on the same evidence you would ask for to add it and after seeing that it still closes. The two keep the balance, and that way, over time, the method interrupts you less.

Every change has an author who answers for it, the person who owns it: the AI never answers for you.

Each principle, with its reason and its evidence, is in The seven principles.

The four moments

The principles say what makes the method work. The moments say when each thing happens: one at the start, which you come back to when the blueprint falls short, two on every change, and the last when something slips through, repeats or isn't needed.



the four moments · 1 at the start, 2 on every change, 1 when something slips through or isn't needed

Moment	When	What happens
1. Prepare	At the start of the project, and again when the blueprint falls short: there is something new to	It has two parts. First the blueprint is made: you decide what it is about, what is delicate, which gates are needed and with which checks, the constitution and what context each gate carries, and you can start from ready-made

Moment	When	What happens
	classify, or a gate is missing, isn't needed or is badly defined	rules. Then the gates the blueprint has defined get built. You build them, or your team does, with the agents and tools you use, and you try to beat them on purpose. An engine can make it easier, but it doesn't take you out of it: this is the most delicate part of the method. Until they hold, no change that has to go through them starts.
2. Brief	On every change, before it starts	What has to pass is fixed, along with a first estimate of the risk, from the base in the blueprint, with your approval if needed. For the checks that ask for them, like the tests that start out red, the cases of this change are written before anyone builds. They are written by someone who won't build, they are seen to fail, and they stay out of the builder's reach.
3. Execute	On every change	The agent works and gets through the gates, one by one. If it gets stuck, it stops and asks. At the end, the risk is recalculated from what was touched, and it can only go up. If it goes up, the change goes back to Brief and the checks of the new level are required.
4. Harden	When something slips through or repeats, and when a gate stops adding anything	Cracks become checks, and what slipped through becomes a question on the reviewer's exam. What no longer adds anything is relaxed or retired. Both change the blueprint, so you go back to Prepare and you approve them, or your team does.

They are not four meetings or four documents. Each moment, with its example, is in The four moments.

How to know if a gate is valid

Before you accept a gate, ask yourself four questions. If any answer is no, it isn't a gate yet:

- Was it decided before the agent started?
- Is it checked without trusting what the agent says?
- Is it out of the agent's reach, so that touching it does the agent no good?
- Have you seen it reject something done wrong, real or planted on purpose?

Checking from the same machine the agent works on does not count as out of its reach: things can happen in between. The third question is answered with a degree, and there are three, each depending less on the agent obeying. In the rule degree you ask it and read the diff: the gate holds for as long as you look. In the platform degree, protected branches, permissions and credentials that are only its own stop it, and that gets you a good part of the way. In the engine degree an engine enforces it, and it is the only one that gives the full guarantee. Your blueprint says which one you are in, and I explain it in How a gate gets fooled.

The core

Ready-made rules and good practices can be adapted to your project. This can't: it is what no engine or configuration can take away. Any engine that claims to apply GDD has to guarantee these points.

- The agent doesn't touch the blueprint or the checks.
- The possible damage is bounded.
- Some decisions are made only by a person.
- Nothing is left going in circles, and nothing happens unseen.

The detail of each point is in The core.

What GDD is not

- **It is not an engine.** It says what makes a gate valid, not which ones you need or how to build them. Meeting the whole core takes an engine, but not a specific one.
- **It is not reading everything.** It isn't looking at nothing either: you look where it matters.
- **It is not ceremony.** If almost everything comes out red, the blueprint is wrong, and if a gate no longer adds anything, it gets retired. If every change feels like a chore, something is unnecessary.
- **It is not a guarantee.** Opening every gate doesn't mean it's right. That's why cracks get hardened and rollbacks exist.
- **It is not spec-driven development (SDD) under another name.** There are scenarios, but short, readable and executable. The gate is what counts, not the document.
- **It is not TDD under another name.** TDD can be part of it, but GDD also decides who writes what, how much is required depending on the risk, and what happens to whatever slips through.
- **It is not only for software.** It is meant for any project, though so far it has been tested on software. Outside it there are fewer machines and more people, and production is where what you do already reaches someone.

How it is measured

A method that demands facts from agents can't ask you for an act of faith: it has to show that it reduces your load, catches what matters, earns back what it costs and scales. If it can't show that, it gets simplified.

Those same facts tell you what to improve in your blueprint, and let you compare it with how you worked before and with other ways of working.

This version doesn't bring its own data yet. Today it rests on my experience and on what other teams publish. GDD's own data will come in a later version.

Where it comes from

GDD doesn't set out to invent anything: it standardizes what has worked for me and what has worked for others. What it adds is a shared language, a test for whether a gate is valid, and a path that links risk and hardening.

Where to go next

GDD has its own language, but it isn't hard to apply: you can start today with what you already have. Meeting the whole core takes an engine: T-minus, which is in development, or another one that meets it.

If you want	Read
To understand the idea, in a minute	The idea
To walk through GDD step by step with an online shop	Hands on
To understand each part: pieces, roles, principles and moments	The pieces and the roles, The seven principles and The four moments
To try it today, with a first gate ready to copy	Start today
To see the path with all the gates, and what happens when something goes wrong	The whole path

If you want	Read
To know what an engine has to guarantee, and how it is measured	The core and How it is measured
To use an engine that enforces GDD	T-minus
To start from ready-made rules for your area	The kits
To learn how to make sure the principles hold in your project	Good practices
To know where each idea comes from, with all the sources	Where it comes from
To read the story of why GDD was born	Article
To read it all in order	Documentation

This manifesto is the reference. If T-minus, another engine or the documentation contradicts this manifesto, the manifesto prevails. Within the documentation, everything else takes precedence over good practices.